INFORME TÉCNICO BREVE – REDSEGURA

Proyecto: RedSegura – Árboles y grafos aplicados a una red de atención de emergencias
Asignatura: Estructura de Datos II
Autor: Luis Gabriel Rivas Díaz
Modalidad: Individual
Tecnología: Java 17, Maven y Apache NetBeans
Docente: Jaider Enrique Reyes Herazo
Periodo: 2026

Corporación Universitaria
Antonio José de Sucre

1. Introducción

RedSegura es una aplicación de consola desarrollada en Java 17 que representa una red de movilidad entre universidades, centros de salud, nodos de transporte y puntos de encuentro de Sincelejo.

El proyecto aplica estructuras de datos como árboles binarios de búsqueda, árboles AVL y grafos mixtos. También implementa algoritmos de búsqueda, recorridos y cálculo de rutas mínimas.

Los tiempos utilizados corresponden a datos didácticos de simulación y no representan información oficial de tránsito o respuesta a emergencias.

2. Decisiones de diseño

El proyecto se organizó mediante paquetes para separar responsabilidades y facilitar el mantenimiento del código.

Las principales decisiones fueron:

- **Árbol binario de búsqueda:** se utilizaron identificadores ordenados para evidenciar el comportamiento de un ABB cuando sus elementos se insertan secuencialmente.
- **Árbol AVL:** se implementaron las rotaciones LL, RR, LR y RL para conservar el equilibrio después de las inserciones y eliminaciones.
- **Grafo mixto:** se implementó una estructura capaz de representar conexiones dirigidas y no dirigidas, conservando identificadores, etiquetas y pesos.
- **Representaciones:** se construyeron lista de adyacencia, matriz de adyacencia ponderada y matriz de incidencia.
- **Recorridos:** se implementaron BFS mediante cola y DFS mediante recursión, utilizando control de vértices visitados.
- **Dijkstra:** se utilizó `PriorityQueue`, relajación de distancias y un mapa de predecesores para reconstruir las rutas.
- **Integración:** se utilizó `RedSeguraCore` como fachada para comunicar la aplicación con las estructuras y algoritmos mediante métodos públicos.
- **Validaciones:** se implementaron excepciones personalizadas para controlar datos inválidos, duplicados y elementos inexistentes.

3. Análisis de complejidad

Estructura o algoritmo	Complejidad temporal	Complejidad espacial
ABB	O(h), promedio O(log n), peor caso O(n)	O(n)
AVL	O(log n) garantizado para operaciones principales	O(n)
Recorridos de árboles	O(n)	O(h)
Lista de adyacencia	O(V + E) de construcción	O(V + E)
Matriz de adyacencia	$O(V^2 + E)$ de construcción	$O(V^2)$
Matriz de incidencia	O(V × E) de construcción	O(V × E)
BFS	O(V + E)	O(V)
DFS	O(V + E)	O(V)
Dijkstra con PriorityQueue	O((V + E) log V)	O(V + E)

Donde V representa el número de vértices, E el número de conexiones, n el número de nodos del árbol y h su altura.

4. Resultados de control

El conjunto de datos contiene 12 vértices y 18 conexiones.

Se verificaron los siguientes resultados:

Q1. Búsqueda de Hospital Universitario

El lugar `S05` fue encontrado en 5 pasos en el ABB y en 4 pasos en el AVL.

Q2. Rotaciones AVL

Se demostraron las cuatro rotaciones: LL, RR, LR y RL. La raíz final del ejemplo de control fue 2 y el árbol quedó balanceado.

Q3. Representaciones del grafo

Se generaron correctamente la lista de adyacencia, la matriz de adyacencia y la matriz de incidencia.

Q4. Recorridos

Desde `S02` se obtuvieron los siguientes resultados:

- BFS: S02, S01, S06, S07, S09, S05, S08, S10, S12, S03, S04, S11.
- DFS: S02, S01, S07, S09, S05, S10, S08, S12, S03, S04, S11, S06.

Los órdenes son diferentes porque BFS explora por niveles y DFS profundiza en una rama antes de retroceder.

Q5. Ruta mínima

La ruta mínima desde `S02` hasta `S08` fue:

`S02 → S01 → S07 → S08`

Costo total: **24 minutos**.

Q6. Bloqueo de conexión

Al bloquear temporalmente `R16`, el algoritmo encontró una ruta alternativa:

`S02 → S01 → S09 → S10 → S08`

El costo se mantuvo en **24 minutos**.

Q7. Manejo de errores

La consulta del vértice inexistente `S99` produjo una excepción controlada y el programa continuó su ejecución.

5. Pruebas y validación

Se ejecutaron pruebas unitarias mediante JUnit 5 utilizando Maven.

Resultado registrado:

- Pruebas ejecutadas: 72.
- Fallos: 0.
- Errores: 0.
- Pruebas omitidas: 0.
- Resultado: `BUILD SUCCESS`.

También se verificó la compilación y generación del archivo JAR mediante `mvn clean package`, así como la ejecución de la aplicación desde el JAR generado.

6. Conclusiones

El desarrollo de RedSegura permitió integrar estructuras de datos y algoritmos fundamentales de la asignatura Estructura de Datos II en una aplicación funcional.

La comparación entre ABB y AVL permitió observar la importancia del equilibrio de los árboles. Las representaciones del grafo facilitaron el análisis de las conexiones, mientras que BFS, DFS y Dijkstra permitieron recorrer la red y calcular rutas mínimas.

Las pruebas unitarias y las consultas de control permitieron verificar el comportamiento de los componentes implementados y el manejo de situaciones especiales.

El proyecto cumple con los resultados funcionales comprobados y dispone de código fuente, pruebas, documentación y repositorio público para su revisión académica.

Corporación Universitaria
Antonio José de Sucre